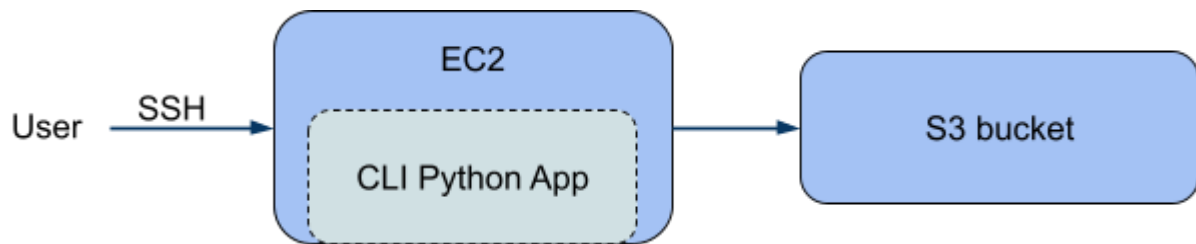


Architecture diagram:



S3 was selected because there is no need for high speed IO operations, the objective is the storage itself.

EC2 was selected because the virtual machine makes it simple to set up with python based tools, with high control on the environment.

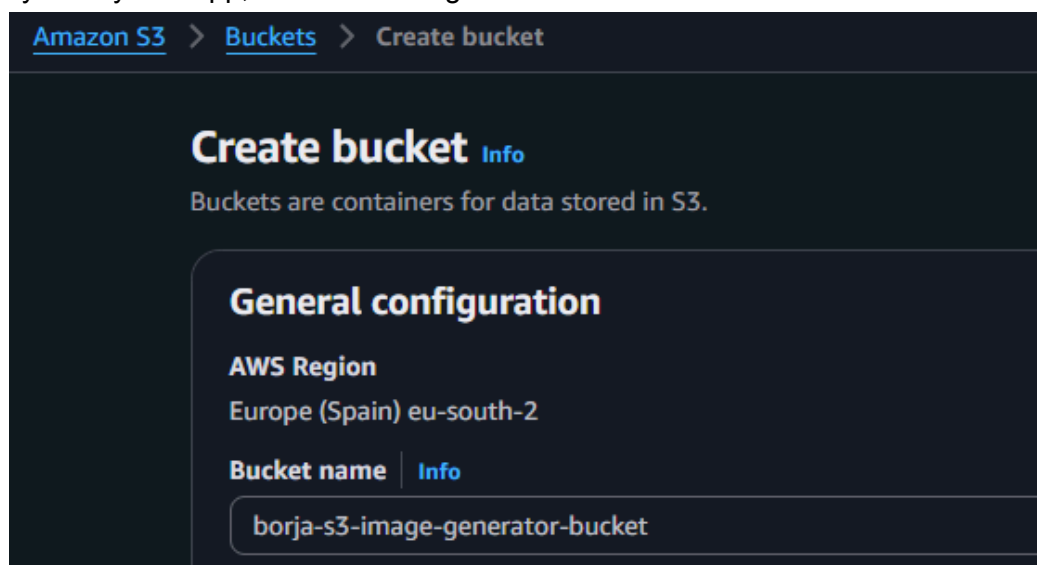
- If changes need to be done to the Python app or its environment EC2 allows for a simple and quick update of the instance
- If the app grows, it allows horizontal scaling, adapting to current traffic dynamically

Setting AWS Services:

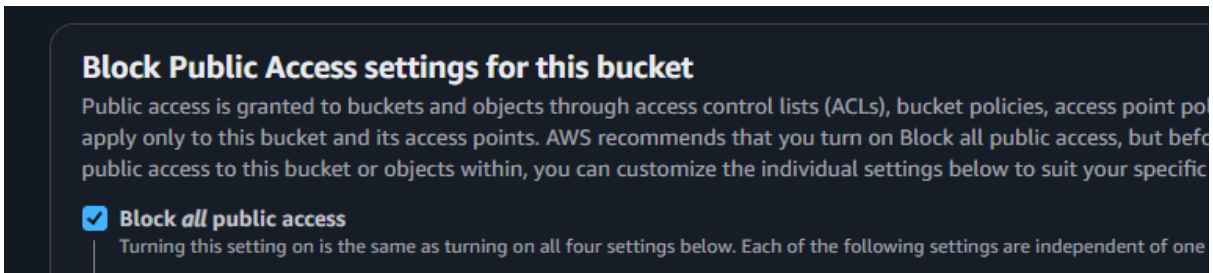
Note: Identifiers such as bucket names are simple examples and not tied to any active service at the time this document is being made. IP's have been partially censored

Setting S3:

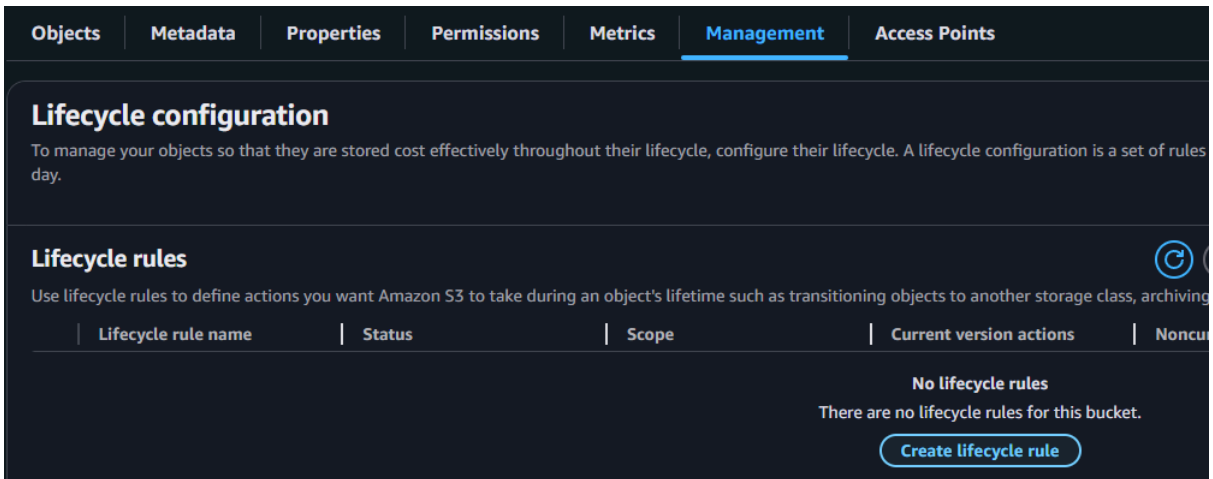
We create the bucket in the S3 service corresponding section. The name will be used both by the Python app, and IAM settings



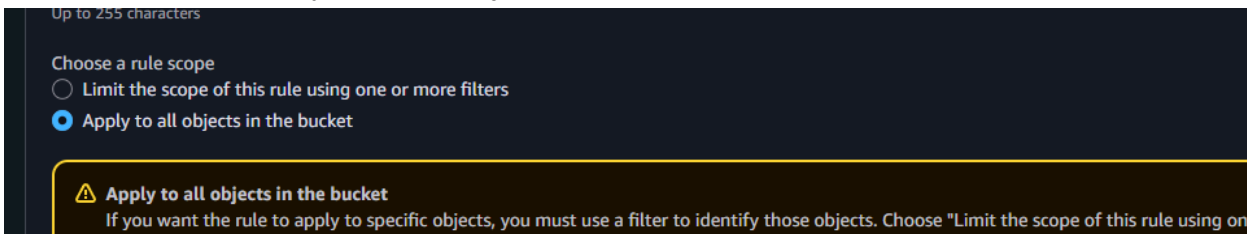
In this case the app will be used through SSH and we block public access



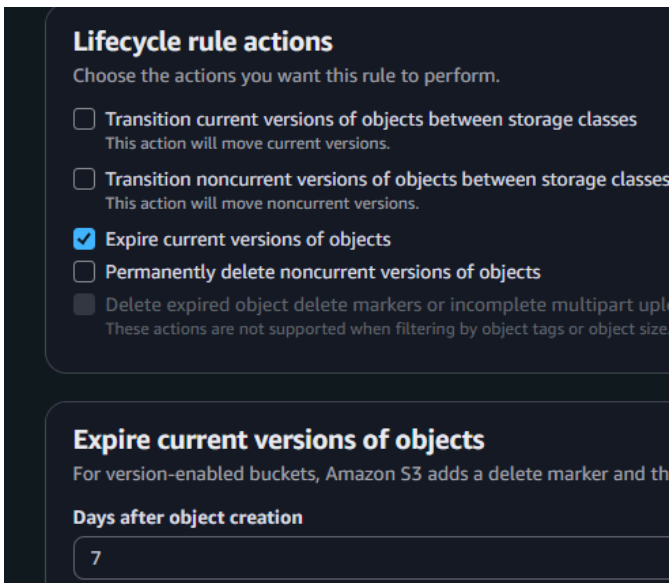
Once created, we go into the bucket, we click on Management section, and we enter “create lifecycle rule”



And we change settings so files are removed after 7 days, to avoid unnecessary storage usage. Versioning is already disabled by default, so this is enough to periodically remove files. We choose to apply rule to all objects



And modify fields in Lifecycle rule options:



Setting EC2:

We go into the EC2 service section and launch an instance. We select a machine adapted to our needs.

For this project, the Python environment is simple, and the files generated are small and quickly moved to S3. So we need little disk space. We also need little RAM. A simple free tier machine is chosen, though for a different Python app, or for multiple concurrent users this would have to be reevaluated carefully.

The screenshot shows the 'Amazon Machine Image (AMI)' selection interface. At the top, it displays 'Amazon Linux 2023 kernel-6.1 AMI' with details: 'ami-0a0afcb8a0ca5bda6 (64-bit (x86), uefi-preferred) / ami-03a14e47332cda85a (64-bit (Arm), uefi)' and 'Virtualization: hvm ENA enabled: true Root device type: ebs'. A 'Free tier eligible' badge is visible. Below this is a 'Description' section stating: 'Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.' The specific AMI is 'Amazon Linux 2023 AMI 2023.10.20260302.1 x86_64 HVM kernel-6.1'. A table below lists attributes: Architecture (64-bit (x86)), Boot mode (uefi-preferred), AMI ID (ami-0a0afcb8a0ca5bda6), Publish Date (2026-03-03), and Username (ec2-user). A 'Verified provider' badge is also present. The 'Instance type' section is expanded, showing 't3.micro' as the selected instance type, which is 'Free tier eligible'. Details include 'Family: t3', '2 vCPU', '1 GiB Memory', and 'Current generation: true'. Pricing information is shown: 'On-Demand Ubuntu Pro base pricing: 0.0149 USD per Hour' and 'On-Demand Windows base pricing: 0.0206 USD per Hour'. An 'All generations' toggle is also visible.

We leave SSH as the only possible access for the instance

The screenshot shows the 'Firewall (security groups)' configuration screen. It explains that a security group is a set of firewall rules that control traffic. Two options are available: 'Create security group' (selected) and 'Select existing security group'. Below, it states: 'We'll create a new security group called 'launch-wizard-2' with the following rules:'. Three rules are listed: 'Allow SSH traffic from' (checked, 'Helps you connect to your instance', source 'Anywhere' 0.0.0.0/0), 'Allow HTTPS traffic from the internet' (unchecked, 'To set up an endpoint, for example when creating a web server'), and 'Allow HTTP traffic from the internet' (unchecked, 'To set up an endpoint, for example when creating a web server'). A yellow warning box states: 'Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.' The 'Configure storage' section is partially visible at the bottom, showing '1x 8 GiB gp3' and 'Root volume, 3000 IOPS, Not encrypted'.

We also generate a key pair, with the corresponding SSH key file to be able to log into the EC2 instance. In this case, for Windows 11 on local computer I use .pem file format.

Key pair name
Key pairs allow you to connect to your instance securely.

ImageGenSSHKey

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

Warning: When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#)

[Cancel](#) [Create key pair](#)

In the advanced settings, we also write a script to set up the Python environment in the machine at instance creation. In this case, the last two commands failed since pip wasn't properly installed, and I had to set them up manually later on the instance itself.

If the project was done again, this script would have to get tested and properly fixed.

User data - optional | [Info](#)

Upload a file with your user data or enter it in the field.

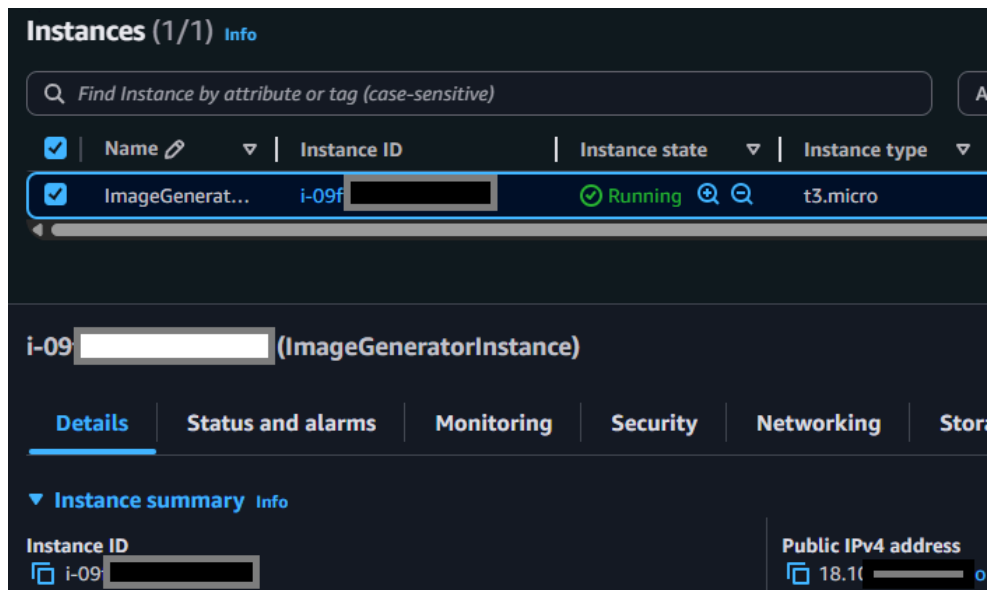
[Choose file](#)

```
#!/bin/bash

sudo yum update -y
sudo yum install -y python3
pip3 install --upgrade pip

pip3 install boto3 pillow
```

After creating the instance we can search for its IPv4 (lower right in the image), which will be used to access it with SSH.



Once the instance is running, and possibly setting up permissions for the key to be used in Windows locally, we upload the Python App with the key we generated. We use the username in the instance (**ec2-user** the default one in this case) , and the IP to connect to the instance.

```
PS C:\Users\borja\Desktop\python\ImgGenerator> scp -i .\ImageGenSSHKey.pem *.py ec2-user@18.1...:/home/ec2-user/
aws_file_manager.py 100% 746 3.3KB/s 00:00
image_generator.py 100% 1948 14.8KB/s 00:00
main.py 100% 1469 5.3KB/s 00:00
```

And then we enter the instance itself

```
PS C:\Users\borja\Desktop\python\ImgGenerator> ssh -i .\ImageGenSSHKey.pem ec2-user@18.1...
```

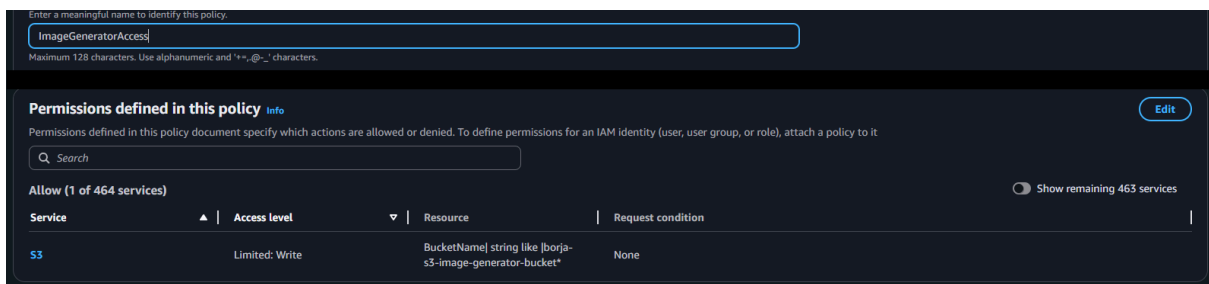
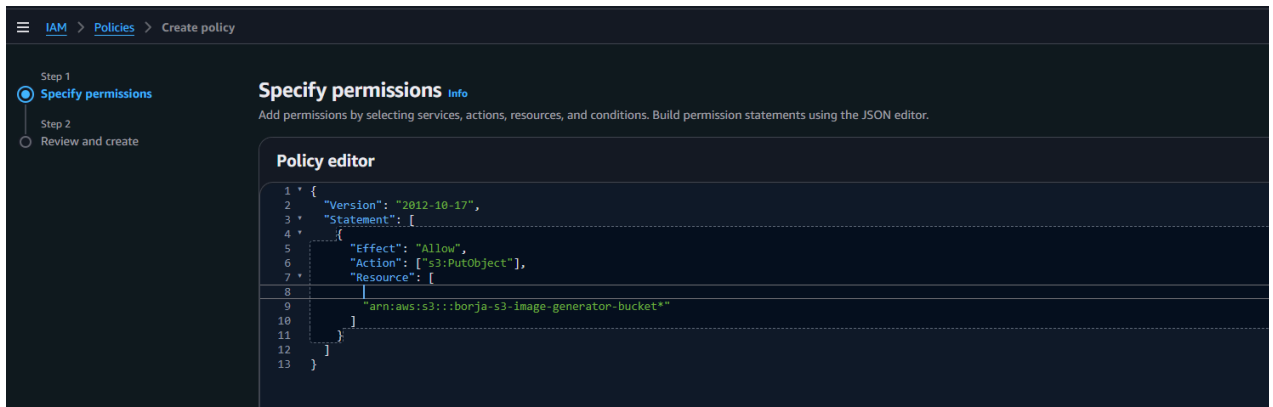
And we install missing dependencies for Python directly on the shell:

- sudo yum install python3-pip -y
- pip3 install pillow boto3

Setting up permissions with IAM:

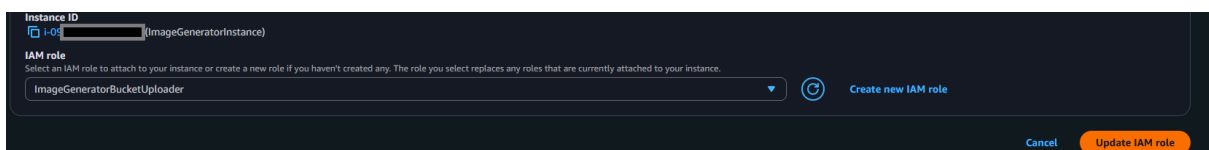
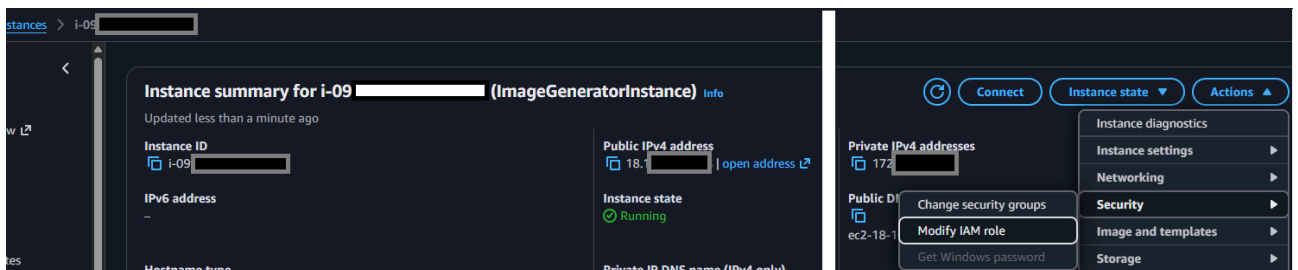
Before setting up a role for EC2 to access S3 bucket, we'll create a specific policy. For this we need the ARN of the S3 bucket, which can be located in the specific bucket section of the AWS website.

Once we have the ARN, we use the policy editor :



And then we create a role that affects EC2 instances and includes this policy.

Lastly, we go into the website section of the relevant EC2 instance we want to assign this role to, and we do it as is shown in the following image.



Testing and results:

We connect to the EC2 instance with SSH and get into the linux shell like we did before, and inside that shell we test the python app:

```
[ec2-user@ip-172-... ~]$ python3 main.py --size 30 --img-pattern 4
/home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting
, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/pyth
warnings.warn(warning, PythonDeprecationWarning)
[ec2-user@ip-172-... ~]$ python3 main.py --size 100 --img-pattern 7
/home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting
, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/pyth
warnings.warn(warning, PythonDeprecationWarning)
[ec2-user@ip-172-... ~]$
```

We get warnings because of the Python version , but if we check the bucket it has the images we generated. We can download them and check they are as we expected

borja-s3-image-generator-bucket

Info

Objects

Metadata

Properties

Permissions

Metrics

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3](#)

Q Find objects by prefix

☐	Name	Type
☐	 2026_03_07_22_43_33_img.png	png
☐	 2026_03_07_22_44_00_img.png	png

